

Frontend Intern Assessment — Mini Lead CRM

Company: Superleap

Role: Frontend Engineering Intern

Time Limit: 72 hours from receipt

Submission: Send your GitHub repository link to the HR you are in contact with

Overview

Build a single-page web application for a Lead Management CRM. A lead represents a potential customer moving through a sales pipeline. Your app should let users browse, search, create, edit, delete, and transition leads through defined stages — with the polish, accessibility, and state-handling we'd expect in a real product.

You do **not** need to build a real backend. We provide a seed JSON file and a one-line `json-server` setup (instructions below). You may also use MSW, an in-memory store, or any mock layer of your choice — but the contract must match the spec.

This assessment has three levels. **Level 1 is required.** Levels 2 and 3 are bonus — they won't hurt you if skipped, but completing them well will stand out. A clean, polished Level 1 is a stronger submission than a broken attempt at all three.

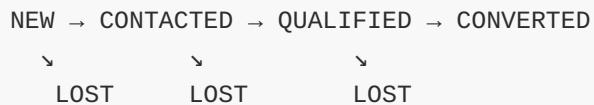
Data Model

A `Lead` has the following fields:

Field	Type	Notes
<code>id</code>	UUID/int	Auto-generated, unique
<code>name</code>	string	Full name of the lead (required)
<code>email</code>	string	Must be a valid email format (required)
<code>phone</code>	string	Optional
<code>status</code>	enum	One of: <code>NEW</code> , <code>CONTACTED</code> , <code>QUALIFIED</code> , <code>CONVERTED</code> , <code>LOST</code>
<code>source</code>	string	e.g., "website", "referral", "campaign" (optional)
<code>created_at</code>	timestamp	Auto-set on creation
<code>updated_at</code>	timestamp	Auto-set on every update

Status Transitions

Not all status changes are valid. Your UI must respect these rules:



- A lead starts as **NEW**.
- A lead can move forward one step at a time: **NEW → CONTACTED → QUALIFIED → CONVERTED**.
- A lead can move to **LOST** from any status except **CONVERTED**.
- Once a lead is **CONVERTED** or **LOST**, it's done — status can't change anymore.

Important: invalid transitions should not even be offered in the UI (e.g., don't render a "Mark as CONVERTED" button on a **NEW** lead). These rules must be enforced *visually*, not just guarded behind error toasts.

Mock API

We'll provide a **seed.json** file with ~50 sample leads.

This gives you **GET/POST/PUT/PATCH/DELETE /leads** for free. You're free to layer your own mock (MSW, in-memory) if you prefer — just keep the same routes so the contract is consistent.

Mock server files: <https://drive.google.com/file/d/1i7N-ekBmeQr1s2v2Q7g530osi-yKwBOB/view?usp=sharing>

Steps to run mock server in included README.

Design reference: <https://drive.google.com/file/d/1riAuDyrBwZCBLx3lIhuSsOrlgQ5W1qfT/view?usp=sharing>

Treat as inspiration only — not required to match exactly. Free to improve on it. Bonus points for better.

Level 1 — Core CRUD UI + Status Rules (Required)

Build a **/leads** route that contains:

List view

- A table or list showing each lead's name, email, status (as a colored badge), source, and last-updated time.
- A **status filter** (visible UI control) — show only leads in selected statuses.

- A **search box** that filters by name or email (client-side filtering is fine).
- Proper **empty**, **loading**, and **error** states. Skipping these is an instant signal.
- Each row should have actions: View, Edit, Delete.

Create / Edit lead

- A form (modal or dedicated page — your call).
- Inline validation:
- `name` required.
- `email` required, must look like a valid email.
- Submit button disabled while invalid.
- Show server errors (e.g., 400 from the mock) gracefully — not as a raw JSON dump.

Delete

- Confirmation step before destructive action.
- Optimistic removal with rollback on failure (or a clear loading state — your choice, but be deliberate).

Status transitions

- A dedicated UI (button, dropdown, menu — your call) for changing status.
- Only **valid next statuses** are shown for a given lead.
- Leads that are `CONVERTED` or `LOST` should clearly show that the status can no longer be changed.

Routing

- URLs should be deep-linkable. Refreshing on `/leads/:id/edit` should still work.
- Use any router (React Router, TanStack Router, etc.).

Requirements summary

- Validate inputs client-side; don't rely solely on the mock to reject them.
- Every async action has loading and error states.
- Use semantic HTML: a table is a `<table>`, a button is a `<button>`.

Level 2 — Kanban Board View (Bonus)

Add a `/board` route showing leads as draggable cards across status columns.

Column	Contents
NEW	All NEW leads
CONTACTED	All CONTACTED leads
QUALIFIED	All QUALIFIED leads
CONVERTED	All CONVERTED leads (locked)
LOST	All LOST leads (locked)

Requirements

- **Drag-and-drop** to change a lead's status.
- **Invalid drops** (e.g., dragging a **NEW** lead onto **CONVERTED**) should be rejected with clear visual feedback — the card snaps back, an error toast or inline message explains why, and no API call is made.
- **Optimistic updates**: the card moves immediately on a valid drop; if the (mock) API fails, revert with feedback.
- **Filters and search persist** between `/leads` and `/board` (via URL params).
- The board should stay readable when a column has 50+ cards (think about scrolling/density).

You may use a DnD library (`@dnd-kit` , `react-beautiful-dnd` , `react-dnd`). If you do, justify the choice in the README.

Level 3 — Bulk Actions + Performance (Bonus)

Bulk operations

- Add row selection (checkboxes) to the list view.
- A bulk-action bar appears when ≥ 1 lead is selected.
- Bulk actions: **delete**, **change status**.
- For bulk status change, only show transitions valid for *every* selected lead. (E.g., if the selection contains a **CONVERTED** lead, no bulk transition is allowed and the UI should explain why.)
- After a bulk action, show a result summary: X succeeded, Y failed, with per-lead error reasons.

Performance

- The list view should stay smooth with **5,000+ leads** in the dataset.
- Solutions: virtualization (`@tanstack/react-virtual` , `react-window`), pagination, or a hybrid. Document your choice.

- Search and filter should remain responsive (debouncing, memoization, or moving work off the main thread — pick what fits).

URL state

- Filter, search, sort, and selected view (list/board) should be reflected in the URL so a user can share or bookmark a filtered view.

Submission Checklist

Your repository must include:

- **Source code** with a clear project structure.
- **README.md** with:
 - **Tech stack** chosen and why (framework, state management, styling, mock API — 2-3 sentences each).
 - **Setup steps** to run locally (ideally one or two commands).
 - **Design decisions** section covering:
 - How you organize component, state, and async logic.
 - How you enforce the status rules in the UI.
 - What you'd do differently for offline support or concurrent edits (two users editing the same lead).
 - What you'd improve given another week.
- **AI usage note** — a short paragraph on how you used AI tools, what you accepted, and where you intentionally wrote things by hand or rejected suggestions. (We're not penalizing AI use; we want to see judgment.)
- **.gitignore** — no `node_modules`, `.env`, build outputs.
- **A 1-3 minute screen recording** (Loom, or any link) walking through the app: a quick demo of the major flows. This lets us see the *feel* of your UI without cloning.

What We're Looking For

We will evaluate your submission roughly in this order:

1. **Correctness** — Does the app work? Do invalid status transitions stay invalid?
2. **UI polish** — Empty / loading / error states, transitions, focus management, sensible spacing and hierarchy. Not "pretty" — *considered*.
3. **State management** — Clean separation between server state, UI state, and form state. No prop-drilling soup.
4. **Component design** — Reusable, single-responsibility components.
5. **Async handling** — Every fetch has explicit loading and error states; nothing swallows errors silently.

6. **Code organization** — Easy for a teammate to find things.
7. **Smart use of libraries** — Picking the right tool, not pulling in 50 dependencies for a small app.

For bonus levels, we additionally look at:

- **Kanban interactions** — Smooth DnD, sensible feedback on invalid drops.
- **Bulk action UX** — How clearly you communicate which actions are valid and what happened after.
- **Performance choices** — Sensible approach for the dataset size, with reasoning in the README.

Rules

- Framework: React or NextJs — your choice.
- Any state library or none. Justify in README.
- Any styling approach: Tailwind, CSS Modules, vanilla CSS, styled-components, or a UI library (shadcn, Mantine, Chakra, MUI). UI libraries are fine, but heavy reliance without customization is visible.
- You may use AI tools for assistance, **but you must understand and be able to explain every line of your code** in a follow-up discussion. We will ask.
- Do not over-engineer. We'd rather see a clean, working Level 1 than a broken attempt at all three levels.
- If you have questions, email us. Asking good questions is a positive signal.

FAQ

Q: Do I need to deploy this? A: No. We will clone your repo and run it locally. Deploying is appreciated but not required.

Q: Do I need authentication? A: No. Keep it simple — no auth, no users.

Q: Can I use a UI component library like shadcn or MUI? A: Yes. But if your entire submission is unmodified library components, we won't see much of *you*. Customize, compose, and show your taste.

Q: Can I use TypeScript? A: Strongly preferred. You don't have to, but real codebases use it and we'd like to see how you model types.

Q: What if I can only finish Level 1? A: That's completely fine. A polished Level 1 with a thoughtful README is a strong submission.

Q: Can I add features beyond the three levels? A: Sure, but only after Levels 1-3 are solid. Things like dark mode, animations, undo/redo, or per-lead activity timelines are nice but not required.

Good luck. We look forward to reviewing your work.